

Four Hands-On Builds for AI Architecture, Data and Security

Four independent, end-to-end systems built on an Azure virtual machine with GitHub Copilot — each one turning an architecture decision into something that runs, fails, and can be measured.

Companion sessions	Module 1 — Architecture patterns and selection · Module 2 — Enterprise reference architecture · Module 3 — Storage layers and data engineering · Module 4 — Secure AI architecture
Companion demonstrations	Demo 1 — Industrial Service Knowledge Copilot · Demo 2 — Predictive Maintenance and Anomaly Triage
Format	Teams of four. Each task is self-contained. Tasks 3 and 4 share a document corpus, but Task 4 ships with its own if you need it.
Toolchain	Azure Linux VM · GitHub Copilot (agent mode) · Python 3.11 · FastAPI · Streamlit · Azure OpenAI chat and embedding deployments
Every task ends with	A running service on a URL the facilitator can call, and a decision your team defends aloud — with the measurement that supports it.

The rule that governs all four tasks. Yesterday you learned that you cannot improve what you cannot measure. Today the same rule applies to architecture. An architecture diagram is a claim about how a system behaves — under load, under failure, under an access rule. A claim you have not tested is a drawing. Every task today ends with the system doing the thing the diagram promised, in front of someone, or with an honest statement of where it did not.

Task	Build	Where	What it makes undeniable
1	Pattern Selection Bench Real compressor telemetry	Lab 1	With four labelled failures you cannot train a supervised model, and — more importantly — you cannot evaluate one. Anomaly detection asks a question your data can answer.
2	The Nineteen Layers Chaos drill on a live service	Lab 2	A PoC has no retries, no timeouts and no fallbacks — and does not need them, until it does. Retry without timeout makes an outage worse, not better.
3	Ingestion Architecture Documents, telemetry, feedback	Lab 3	Two embedding model versions in one index degrade retrieval silently. Nothing errors. No alert fires.
4	Secure Retrieval, Three Roles Cross-team red-team round	Lab 4	Retrieve-then-filter is not access control. The restricted content already shaped the answer.

The four tasks run inside the Day 2 lab sessions and together produce every one of the seven Day 2 assignment deliverables — see the closing page. Readouts are given at the start of the session following each lab, so they do not consume lab time.

Before you start — environment and ground rules

One VM per team

If you built the Day 1 environment, you need only the additional packages in section 2 and the embedding deployment in section 3. If you are starting fresh, allow enough time to complete all six sections before the first lab.

1 · The virtual machine

Size	Standard_D4s_v5 — 4 vCPU, 16 GB RAM. Task 1 loads a 208 MB CSV; nothing today needs a GPU.
Image	Ubuntu Server 24.04 LTS · 64 GB Premium SSD
Networking	SSH (22), Streamlit (8501) and FastAPI (8000) inbound, source restricted to the training-room public IP . Never leave these open to the internet — today of all days.
Naming	vm-acday2-team<n> — the facilitator calls these by name during the readout, and Task 4 requires teams to reach each other's services.

```
az group create -n rg-acday2 -l westeurope

az vm create -g rg-acday2 -n vm-acday2-team1 \
  --image Ubuntu2404 --size Standard_D4s_v5 \
  --admin-username azureuser --generate-ssh-keys

# Restrict to the training-room egress IP. Do NOT use `az vm open-port` — it opens the port to *.
az network nsg rule create -g rg-acday2 --nsg-name vm-acday2-team1NSG \
  -n allow-lab --priority 900 --access Allow --protocol Tcp \
  --destination-port-ranges 8000 8501 --source-address-prefixes <training-room-ip>/32
```

2 · The toolchain

```
ssh azureuser@<public-ip>
sudo apt update && sudo apt install -y python3-pip python3-venv git
python3 -m venv ~/.venv && source ~/.venv/bin/activate
pip install fastapi "uvicorn[standard]" httpx pydantic pyjwt streamlit plotly \
  pandas numpy scipy scikit-learn pyarrow duckdb jsonschema \
  faiss-cpu rank-bm25 sentence-transformers openai python-dotenv
mkdir -p ~/task1 ~/task2 ~/task3 ~/task4
```

Editing. VS Code with Remote-SSH from your laptop, GitHub Copilot enabled, agent mode permitted by organisation policy. Run services with `uvicorn app:api --host 0.0.0.0 --port 8000` and dashboards with `streamlit run app.py --server.address 0.0.0.0`.

3 · Azure OpenAI — two deployments today, not one

Task 3 needs **two different embedding models**, and Tasks 2 and 4 need a chat model. Request all three before the day starts.

```
# ~/.env — ask for one set of deployments per team; a shared one will rate-limit
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_KEY=<key>
AZURE_OPENAI_API_VERSION=2024-10-21
AZURE_OPENAI_CHAT=gpt-4o-mini
AZURE_OPENAI_EMBED_V1=text-embedding-ada-002
AZURE_OPENAI_EMBED_V2=text-embedding-3-small
```

Watch for — and this is the point of Task 3, so do not fix it early. Both of those embedding models are documented at 1,536 dimensions. Confirm it yourself with `len(embedding)` in your first cell. Because the dimensions match, mixing their outputs in one index raises **no error at all**. That is exactly the failure Task 3 is built around.

If you cannot get two embedding deployments, use `sentence-transformers` locally instead: `all-MiniLM-L6-v2` and `all-MiniLM-L12-v2` both output 384 dimensions and are equally incomparable. Everything in Task 3 works offline this way.

4 · Secrets — read this before you write any code today

Module 4 covers secrets management, and Task 4 audits your own system for leakage. Start correctly rather than retrofitting.

- `.env` is in `.gitignore` from the first commit. Check it before the second commit, not after.
- No key in a notebook, a prompt, a log line, a trace, or a Streamlit caption.
- Every service reads configuration from the environment. If a key appears in `git log -p`, you will find it in Task 4 and it will count against you.

5 · Working with Copilot — the discipline that separates the top teams

Copilot will scaffold a FastAPI service, a retrieval pipeline or an ingestion job in minutes. That speed is the reason today's tasks are about what happens when things fail, not about whether you can get them running.

Three rules for Copilot use in these labs.

1. **Never accept a network call without a timeout.** Copilot writes `httpx.post(url, json=...)` with no deadline, because that is what most code on the internet looks like. A call with no timeout is an outage waiting for a slow dependency.
2. **Ask it what happens when this fails.** After it produces a component, prompt: *"List every external dependency here and what this code does when each one is slow, unavailable, or returns an error."* The answer is usually short and alarming.
3. **Read the filter.** In Tasks 3 and 4, whatever else you skim, read the lines that decide which documents a query can reach. That is where today's serious mistakes live.

6 · How your work is scored — identical for all four tasks

Criterion	Points	What full marks looks like
The system runs	15	The facilitator calls the endpoint or opens the URL, and it responds. No screenshots.
The decision is justified against criteria	20	Named criteria from the module, applied in order, with a rejected alternative and why.
The claim is measured, not asserted	20	Every architectural claim has a number behind it — a lead time, a p95 latency, a recall, a leak count.
The failure mode is demonstrated	20	The wrong design and the corrected design shown together, with the mechanism explained in one sentence.
What was deliberately left out	10	Layers or controls excluded on purpose, each with a stated reason. "We never thought about it" scores zero; "we decided not to, because" scores full.
The verdict is actionable	15	An engineering manager could read one screen and know what to build next.

Automatic deduction: minus 10 for any architecture diagram with no corresponding running component or explicit "planned" marking; minus 10 for any external call in your code with no timeout. These are the two habits Day 2 exists to break.

Task 1 — Pattern Selection Bench

Lab 1

MODULE 1

PATTERN SELECTION

GROUND TRUTH AVAILABILITY

ANOMALY DETECTION

GROUP LEAKAGE

REAL DATA

The scenario

Five industrial use cases are on the table and each one has a tension in it. Your team will produce a defensible pattern decision for all five — and then **settle the second one empirically**, because it is the case where engineering instinct and the evidence point in opposite directions.

Case 2 as written: vibration-based failure prediction on 300 compressors, with **eleven labelled failures in three years**. Almost every team's first answer is a supervised classifier. Today you will build one, and discover that the problem is not that it performs badly. The problem is that **you cannot tell whether it performs badly**.

The data — a real air compressor, with real failures

Dataset	MetroPT-3, UCI Machine Learning Repository (ID 791). The Air Production Unit — a reciprocating air compressor — on a metro train, instrumented and monitored continuously.
Size	1,516,948 readings at 1 Hz, February to August 2020. File: <code>MetroPT3(AirCompressor).csv</code> , about 208 MB.
Analogue signals	TP2, TP3, H1, DV_pressure, Reservoirs (all bar) · Motor_current (A) · Oil_temperature (°C)
Digital signals	COMP, DV_electric, TOWERS, MPG, LPS, Pressure_switch, Oil_level, Caudal_impulses
Failures	Four air-leak failures: 18 April · 29–30 May · 5–7 June · 15 July 2020. That is your entire labelled positive set — the real-world version of Case 2.
Why this dataset	It is compressed-air equipment, continuously monitored, with genuinely scarce failure labels. Nothing about the difficulty has been manufactured for the exercise.

```
wget https://archive.ics.uci.edu/static/public/791/metropt+3+dataset.zip
unzip metropt+3+dataset.zip
python -c "import pandas as pd; d=pd.read_csv('MetroPT3(AirCompressor).csv'); \
print(d.shape); print(list(d.columns)); print(d['timestamp'].min(), d['timestamp'].max())"
# Downsample to 1-minute means before doing anything else – 1 Hz is far more
# resolution than an air leak needs, and it makes every later step interactive.
```

Step 1 · Build the pattern decision record for all five cases

Before any modelling, apply Module 1's six-question sequence to each case, in order, and record where each question eliminates something. Produce one **Architecture Decision Record** per case — a short structured document, not prose.

ADR field	Content
Case and decision	The chosen pattern, named from the thirteen.
The three deciding criteria	From the twelve. Not all twelve — the three that actually settled it.
Rejected alternative	One serious alternative, and the specific criterion that ruled it out.
Biggest risk in the choice	What would make this decision wrong, and how you would find out early.
Reversal cost	What it would cost to change this in six months. This is the field teams skip and the one that matters.

Answer the six questions out loud, in order. Data type · does ground truth exist · latency budget · act or recommend · who may see what · what must it integrate with. The first two eliminate whole families of pattern, which is why they come first. If your team reaches question three still holding four candidate patterns, you have not answered questions one and two honestly.

Step 2 · Build the supervised model — the one everybody wants

Label the readings around each of the four failure events as positive, everything else negative, and train a classifier. Split the data randomly by row, as most tutorials do. Report accuracy, precision, recall and ROC-AUC.

Your scores will be excellent. Write them on the dashboard.

The planted defect — group leakage, and it is enormous here. Consecutive readings from the same compressor seconds apart are almost identical. Splitting by row puts near-duplicate readings on both sides of the split, so your test set is effectively part of your training set. The model has memorised four specific failure episodes and is being congratulated for recognising them.

Now split **by failure event** instead — train on three, test on the fourth. That is the only honest split, and there are exactly four ways to do it. Report all four results side by side with your row-split result. This is Day 1's group-leakage row, met again in a place where it is much harder to see.

Step 3 · The measurement that actually settles the case

Forget performance for a moment and ask the prior question: **with four positive events, how precisely could you measure recall even if the model were perfect?** Compute the bootstrap confidence interval on recall over the failure events.

What you will find. With four events, catching three of them gives a recall of 0.75 with a 95% interval running roughly from 0.30 to 0.95. You cannot distinguish a good model from a poor one. You cannot show an improvement. You cannot fail a model that deserves to fail.

That is the answer to Case 2, and it is not "supervised learning performs worse here". It is "**supervised learning cannot be evaluated here**" — and an architecture you cannot evaluate is one you can never improve, which is precisely the position Day 1 spent eight hours diagnosing.

Step 4 · Build the pattern the data can actually support

Anomaly detection asks a different question — *is this behaviour unusual for this asset?* — and it needs no labelled failures at all. Train on **February and March only**, which the dataset documentation designates as the normal-behaviour period, then score everything afterwards.

- Build at least two detectors: a **statistical control-limit** detector on rolling windows — yesterday's chart, reused — and one of isolation forest, one-class SVM, or autoencoder reconstruction error.
- Score April to August. Do not use the failure dates during fitting, at any point, for anything.
- For each of the four failures, measure **lead time**: how long before the documented failure did the anomaly score first cross your threshold and stay crossed?
- Convert the false-alarm rate into **alerts per engineer per day** across a 300-compressor fleet, exactly as in Day 1 Task 2.

The comparison to put on your dashboard. Two columns. Supervised: an excellent score that is unmeasurable, unimprovable and built on four episodes. Anomaly detection: a lead-time distribution over four independent events, an alert volume you can defend, and a threshold you can tune — without a single labelled failure. That table is the deliverable. It is also the answer to the question the case was really asking.

Step 5 · The bench itself

One Streamlit page. Five ADR cards for the five cases · the signal traces around each failure with your anomaly score overlaid · the honest-versus-leaky supervised comparison · the lead-time chart · the alert-volume calculator · and a **decision panel** that takes the six questions as inputs and names the pattern it implies, so the reasoning is executable rather than remembered.

Acceptance criteria

- ☐ Five ADRs, each naming three deciding criteria, one rejected alternative and a reversal cost.
- ☐ Row-split and event-split supervised results shown together, with the gap explained in one sentence.
- ☐ The confidence interval on recall over four events is reported, and its width is stated as the reason for the decision.
- ☐ Anomaly detector trained on February–March only, with fitting code that never touches the failure dates.
- ☐ Lead time reported for all four failures individually, including any the detector missed.
- ☐ Alert volume stated per engineer per day for a 300-compressor fleet.

Defend your result

"For Case 2 we chose ____ because _____. We rejected ____ because with four labelled events the interval on recall runs from ____ to _____, which means we could never have measured it. Our detector gives ____ hours of lead time at ____ alerts per engineer per day. The biggest risk in this choice is _____."

Stretch. Take the anomaly scores from the four failures and ask which signals deviated first — TP2 and the reservoir pressures behave differently from motor current in an air leak. An anomaly detector that can name the deviating signal gives a maintenance engineer something to act on; one that emits a score alone gives them a number to ignore. Explainability was one of the twelve selection criteria, and this is what it looks like in practice.

Task 2 — The Nineteen Layers

Lab 2

MODULE 2

ORCHESTRATION

RETRY & FALLBACK

REGISTRIES

AUDIT

CHAOS DRILL

The scenario

You have a working proof of concept. It has three components: a script, a model call, and a notebook someone runs. It answers questions correctly and it cannot be deployed, and the gap between those two facts is sixteen layers.

In this lab you will build the naive version deliberately, then add the layers that make it a system — and then the facilitator will break your dependencies while it is running, and you will find out which layers you actually built.

Step 1 · Build the PoC, honestly bad

Write `poc.py`: a single FastAPI endpoint that takes a question, calls Azure OpenAI, and returns the text. No authentication, no timeout, no retry, no logging, no versioning. It should be about forty lines. Do not improve it — this is your current-state baseline and you will measure against it.

COPILOT — THE DELIBERATELY NAIVE BASELINE

Create `poc.py`: a FastAPI app with one POST `/ask` endpoint taking `{"question": str}`. It calls Azure OpenAI chat completions directly and returns the raw text. No auth, no timeout, no retry, no logging, no request id, no model version in the response. This is intentionally a proof-of-concept baseline — do not add production concerns. Then generate `current_state.mmd`, a Mermaid diagram of exactly what this file does, with no components that do not exist in the code.

Run a load of 200 requests against it and record: success rate, p50 and p95 latency, and what a caller sees when something goes wrong. That row is the top of your comparison table.

Step 2 · The layer manifest — decide before you build

Create `layers.yaml` listing all nineteen layers, each marked **present**, **planned**, or **excluded** with a reason. This file is the deliverable that the target-state diagram is generated from, so the diagram cannot claim anything the manifest does not.

Group	Layers	Minimum for full marks today
Access	Interface · API · authentication · authorisation · RBAC	All five present, even if simply
Reasoning	Orchestration · tool integration · model serving	Orchestration present; the other two present or reasoned
Data	Vector · relational · object · graph · feature store	Two present, three reasoned — a graph store you do not need is a legitimate exclusion
Governance	Prompt registry · Skill registry · model registry	Prompt and model registries present
Operations	Monitoring · logging · audit trail	All three present, and separate
Improvement	Feedback loop · human approval	Feedback present; approval present or reasoned

An exclusion is a legitimate answer, and it is worth marks. Not every system needs all nineteen on day one. Every system needs a deliberate decision about each — including the decision to leave one out for now, with a note of why. The gap between "we decided not to" and "we never thought about it" is the entire point of this task.

Step 3 · Build the layers that carry the load

Layer	What must actually work by the end of the lab
API	One entry point. Request validation, a versioned path (<code>/v1/ask</code>), per-caller rate limiting, and a request id returned on every response.
Authentication	A bearer token carrying an identity. A signed JWT with a shared secret is sufficient today — the point is that identity exists and propagates, not that you have federated to an enterprise provider.
Authorisation and RBAC	Distinct from authentication. A role on the token; a policy that decides which operations that role may invoke. A caller can be correctly authenticated and still refused.
Orchestration	The six responsibilities, all six: routing, sequencing, retry with backoff, timeouts , context assembly, and a guardrail hook — even a stub one, so the seam exists.
Model serving	A model version in every response. A response cache keyed on the question plus prompt version. A cheap-model route for short questions and an expensive route for the rest.
Prompt registry	Prompts in versioned files, never inline strings. Every response reports which prompt version produced it.
Model registry	Which model version is serving, what it scored, on which dataset version, approved by whom. A JSON file is a registry if it is versioned and authoritative.
Monitoring · logging · audit	Three separate stores, not one. Monitoring aggregates and is fast. Logging is per-request traces and spans. Audit is append-only, immutable, and answers who asked what under whose authority.
Feedback	A thumbs up or down joined to the request id, stored so that it could later be joined to training data.

The planted defect, and the drill will find it. Copilot will give you retries. It will not give you timeouts. Retry without a timeout does not make you more resilient — it makes an outage three times longer, because each attempt waits indefinitely and you have now queued three of them. Every external call needs a deadline, and the deadline has to be shorter than your own caller's patience.

Step 4 · The chaos drill

Partway through the lab the facilitator will announce the drill. Each team runs a fixed load of 200 requests through their service while four faults are injected in turn. You may not change code during the drill.

Fault injected	How	What a system with the layers does
Model endpoint unreachable	Point the endpoint at a dead host	Retries, backs off, falls back to a cached or degraded answer, tells the caller honestly
Model endpoint very slow	Proxy adding an 8-second delay	Times out well before the caller does, returns a clean degraded response
Rate limit	Endpoint returns HTTP 429	Backs off, queues or sheds load, does not hammer
Retrieval store unavailable	Rename the index directory	Answers without retrieval and says so, rather than erroring

Record, for each fault: **requests served, requests failed, p95 latency, and what the caller actually saw**. Compare every row against the same test on `poc.py`.

The number that wins this lab. Percentage of requests answered usefully while a dependency is down. The naive PoC scores zero on every fault. Do not be surprised if your first attempt scores zero on two of them — that discovery is the lab working.

Step 5 · Diagrams generated from the manifest, not drawn by hand

Generate `current_state.mmd` and `target_state.mmd` as Mermaid, directly from the code and from `layers.yaml`. Colour each layer by present, planned or excluded. A hand-drawn diagram can claim anything; a generated one cannot.

Then produce the two remaining deliverables: a **gap list** ranked by what blocks production first, and a **scalability risk table** naming what breaks at ten times the volume and which component goes first. For the second one, do not guess — run ten times the load and watch.

Acceptance criteria

- ☐ Naive PoC and layered service both running, both measured on the same load.
- ☐ `layers.yaml` covers all nineteen layers with present, planned or excluded — every exclusion carrying a reason.
- ☐ Every external call in the codebase has an explicit timeout. Grep for it in front of the room.
- ☐ Monitoring, logging and audit are three separate stores with different shapes and retention.
- ☐ Chaos drill results for all four faults, against both services.
- ☐ Both diagrams generated from code and manifest, not drawn by hand.
- ☐ Gap list ranked by what blocks production first, and the first component to break at ten times volume — measured.

Defend your result

"Our PoC served ____% of requests during a dependency outage; our layered service served ____%. The layer that made the largest difference was _____. We deliberately excluded _____ because _____. At ten times volume the first thing to break is _____, and we know because we ran it."

Task 3 — Ingestion Architecture

Lab 3

MODULE 3

ELEVEN STORAGE LAYERS

DATA CONTRACTS

LINEAGE

SUPERSESSION

EMBEDDING VERSIONS

The scenario

Three kinds of data arrive by three different routes, and each needs a different answer to the same four questions: where does it live, how is it versioned, who may see it, and how long is it kept?

Data type	Arrives as	The hard part
Technical documents	Published to a document system, superseded periodically	A withdrawn procedure must stop being cited — quickly, and provably
Telemetry	Continuous streams from 300 assets	What is retained raw and for how long; where features live so training and serving agree
Feedback	Ratings, corrections, escalation outcomes	Joining it back to the original request, months later

You will build this as a working pipeline — not a diagram — and in the middle of it you will meet the most expensive silent failure in enterprise retrieval.

Step 1 · The corpus and the gold set

Generate a corpus of **300 service bulletins and maintenance procedures** for a compressor product family, and a **40-question gold set** with the document that answers each. Copilot produces both quickly, and a synthetic corpus is the right choice here because retrieval quality has to be measurable against known-correct answers.

COPILOT — CORPUS AND GOLD SET

Generate a corpus of 300 markdown documents for an industrial air-compressor service organisation. Mix four types: maintenance procedures, troubleshooting guides, service bulletins, and parts catalogue entries. Each has YAML front matter: doc_id, title, doc_type, product_family (one of GA-30, GA-75, ZR-160), revision, issued_date, status (active|superseded), supersedes (doc_id or null), access_level (public|internal|restricted). Make 30 of them supersede an earlier document with genuinely conflicting instructions. Then generate gold_questions.jsonl: 40 questions, each with the doc_id that answers it, covering all three product families and all four document types. Do not embed or index anything yet.

Step 2 · The eleven layers, as real directories

Create the storage layers as actual, separate locations, and write one line per layer stating its key property. A layer you cannot point at does not exist.

```
store/
  01_raw/           # exactly as received. Immutable. Never edited.
  02_cleaned/       # validated, deduplicated, schema-conformant
  03_features/      # computed features – one definition, used by training and serving
  04_embeddings/    # vectors, each carrying the model version that produced it
  05_index/         # rebuildable search structure, with filterable metadata
  06_metadata/      # source, version, access level, dates
  07_ground_truth/  # the gold set, versioned, with provenance per item
  08_feedback/      # joinable to the original request id
  09_audit/         # append-only. Never rewritten.
  10_models/        # artefacts, versioned alongside the data that produced them
  11_prompts/       # prompts, policies and Skill definitions, reviewed like code
```

The test to apply as you go. Pick any object your pipeline creates and name which of the eleven it belongs to. If you cannot, it is probably in three of them inconsistently — and one of the three is somebody's laptop.

Step 3 · Data contracts that fail loudly

Write a JSON Schema contract for each of the three data types and enforce it at ingestion. Then have a teammate deliberately corrupt a batch — rename a field, send a temperature of 4,000 °C, truncate a document mid-sentence, duplicate fifty records — and confirm the pipeline **rejects the batch and says why**.

Implement at least five of the eight quality checks: completeness, validity, consistency, uniqueness, freshness, distribution, referential integrity, label quality.

Why this matters more for AI systems than for ordinary pipelines. A traditional pipeline fails loudly when a schema changes. An AI system absorbs the malformed input, produces a confident and wrong answer, and nobody notices for weeks. Validation at ingestion converts a silent quality failure into a loud pipeline failure — which is exactly the trade you want. A rejected batch is a bad morning; a month of quietly degraded answers is a bad quarter.

Step 4 · The embedding-version experiment — the centre of this lab

Do this in exactly this order and measure at every stage. Everything you need is the gold set from Step 1 and a retrieval metric — recall at 5 is sufficient.

	Do this	Then measure
1	Embed all 300 documents with embedding model v1 . Build the index.	Recall@5 on all 40 gold questions. This is your baseline.
2	Confirm both models output the same number of dimensions.	Note that nothing will error when you mix them.
3	Add 100 new documents embedded with v2 into the same index , as a real team would when a better model is released.	Recall@5 again, on the same 40 questions.
4	Increase the v2 share to 30%, then 50%, re-measuring at each stage.	Plot recall against the proportion of v2 content.
5	Now fix it: record <code>embedding_model_version</code> as metadata on every vector, and either split into per-version indexes or re-embed everything with one model.	Recall@5 restored. State the cost of the re-index in documents and in minutes.

What you are watching for, and why it is dangerous. An embedding is only meaningful relative to the model that produced it. Two models with the same output dimensions produce vectors that sit in *different spaces*, and distances between them are meaningless. No exception is raised. No alert fires. Retrieval simply gets worse, and diagnosing it months later is genuinely difficult because the symptom is diffuse — some questions get worse, most are fine, and nothing in the logs changed.

The architectural rule: **record the embedding model version on every vector, and treat an embedding-model change as a full re-index** — budgeted in time and cost like the migration it is.

Step 5 · Supersession, and why it needs event-driven ingestion

Thirty of your documents supersede an earlier one with conflicting instructions. Build the path that handles it and prove it with a timed test:

1. Ask a question whose answer comes from the soon-to-be-superseded procedure. Record the citation.
2. Publish the superseding document through your ingestion path.
3. Ask the identical question again. Measure **how long** until the system stops citing the withdrawn procedure.
4. State your freshness contract — and whether nightly batch would have satisfied it.

The sentence this step earns you. "For service manuals, nightly batch is fine. For a procedure that has just been withdrawn by a safety notice, it is not — because a system confidently citing a withdrawn procedure is considerably worse than one that finds nothing at all." Choosing between batch, streaming, event-driven and change data capture is not a technology preference. It is the answer to one question: how stale may this data be before the answer becomes wrong?

Step 6 · Lineage — the test the facilitator will run on you

Record, for every answer your system produces: source document ids and revisions, the transformation version that produced the chunks, the embedding model version, the index version, the prompt version, and the model version. Then the facilitator will pick one answer from earlier in the lab and ask you to **reproduce it exactly**. Most teams cannot. Discovering that in a workshop costs an afternoon; discovering it in an audit costs considerably more.

Acceptance criteria

- ☐ All eleven storage layers exist as separate locations, each with its key property stated.
- ☐ Every data object mapped to one layer, with version, access level, retention period and quality check named.
- ☐ A deliberately corrupted batch is rejected, with a message naming the violated contract.
- ☐ Recall plotted against the proportion of mixed-version content, with the fix applied and the recovery shown.
- ☐ Supersession latency measured, and the freshness contract stated.
- ☐ One earlier answer reproduced exactly, from the lineage record alone.

Defend your result

"Mixing two embedding versions cost us ___ points of recall at ___% mixed content, with no error and no alert. We detected it by ___ and fixed it by ___, at a re-index cost of ___. Our freshness contract for safety notices is ___, which rules out ___ ingestion."

Task 4 — Secure Retrieval, Three Roles

Lab 4

MODULE 4

RBAC

ACCESS-FILTERED RETRIEVAL

PER-ROLE EVALUATION

LEAKAGE PATHS

RED-TEAM ROUND

The scenario

One corpus, three roles, and a rule that has to hold every time. This is the AI-specific security problem: get it wrong and every other control you built today is decoration.

Role	May access	Must not access
Service engineer	Maintenance procedures, troubleshooting guides, parts catalogues for certified product families	Design documents, unreleased specifications, commercial terms, other customers' records
Product engineer	Design documents, specifications, test reports, failure analyses	Customer-specific commercial data, personal data in service records
Customer support	Published documentation, warranty terms, the requesting customer's own service history	Internal engineering documents, other customers' data, cost and margin information

Extend the Task 3 corpus with design documents, test reports, warranty terms, and customer-specific service records for three fictional customers. If you did not complete Task 3, generate a 200-document corpus with the same front matter — Copilot does it in one prompt.

Step 1 · Build the wrong design first, and prove it is wrong

Implement **retrieve-then-filter**: search the whole corpus, generate the answer, then remove restricted documents from the citation list. This is what most systems do, and it looks correct from the outside because the citations are clean.

Now demonstrate the leak. Write a question whose answer exists **only** in a restricted design document, and ask it as a service engineer.

What you will see. The system answers. The citations are clean. The content came from a document the user may not see. The restricted material entered the model's context and shaped the answer, and you are relying on the model not to reveal it — which is not a control, it is a hope.

The librarian. A librarian who fetches every book, reads them, summarises them and then tries not to mention the restricted ones has already read the restricted ones to you, in aggregate. A librarian who only searches the shelves you are permitted to use has not. Only the second is access control.

Step 2 · Build the right design

The user's access rights become **part of the retrieval query**. Restricted documents are never candidates, so they cannot influence anything.

- **Metadata schema** on every chunk — not every document, every chunk: `access_level`, `product_family`, `customer_id`, `doc_type`, `status`.
- **Filter predicate** built from the caller's identity and role, applied inside the search call, before any scoring.
- **Identity propagation** — the same authenticated identity from Task 2 reaches the retrieval layer. Not a service account. Not "the AI system".
- **The empty path** — "nothing available to you" is a correct answer, and the system must handle it gracefully rather than treating it as a failure to work around.

Re-run the same question as a service engineer. The correct behaviour is a clean refusal, not a redacted answer.

Step 3 · Test cases — and the negative ones are what count

Write, for each of the three roles, **one query that must return results and one that must return nothing**. Six tests minimum, run automatically, with the expected outcome asserted.

The second test case matters most. Teams write the positive tests naturally — an engineer asks for a maintenance procedure and gets it. They skip the negative ones. The negative tests are what prove the control works, and **a system that never returns "nothing available to you" has probably not implemented access control at all.** If every one of your six tests passes on the first run, check that your negative queries genuinely target restricted content.

Step 4 · Evaluate per role — the step almost nobody has considered

Run your Task 3 gold set three times, once as each role, and report **three separate scores**.

Why this is a requirement and not a refinement. A gold set built with unrestricted access measures an experience no real user has. A service engineer's answer quality is bounded by what a service engineer can reach. If your headline retrieval number was measured as an administrator, it describes nobody — and the difference between the three role-scoped scores is the honest picture of what each population actually receives.

Step 5 · Audit your own system for the four leakage paths

Path	How to check it in your own code
Retrieval returns what the user may not see	Steps 1–3 above. Assert on candidate sets, not only on citations.
The model repeats sensitive content from context	Mask at ingestion, where the system never needs the value. Masking at output is far weaker — the value has already entered the context.
Tool calls send internal data to an external service	List every outbound call your system can make and what payload it carries.
Logs and traces capture sensitive content in plain text	Open the logging and audit stores you built in Task 2. Grep them for restricted document text and for your API key.

The fourth path is the one this room will fail. Observability systems store full prompts and responses by default — that is what makes them useful. Your Task 2 trace log almost certainly contains restricted document text in plain text, readable by anyone with access to the logs, retained for weeks. Find it, then decide what your system should store instead: the document ids rather than the passages, and a hash rather than the text.

Step 6 · The red-team round

Teams exchange service URLs and role tokens. You will be issued a **service engineer** token for another team's system. Your objective is to obtain content that role must not access, **using only legitimate queries** — clever phrasing, indirect questions, asking for summaries or comparisons, requesting a document by name.

Rules for the round. Legitimate queries only. Do not attempt prompt injection, do not attempt to forge tokens, and do not attack the infrastructure — injection is a hands-on lab later in the week and doing it now spoils it. Report every confirmed leak to the team that owns the system, with the query that produced it. You are scored on leaks found in others' systems *and* on leaks found in yours — a team that finds nothing has usually not tried hard enough, and a team whose system survives has built something real.

Acceptance criteria

- ☐ Retrieve-then-filter and filter-then-retrieve both demonstrated, with the leaking question shown answering in one and refusing in the other.
- ☐ Access metadata on every chunk; the filter predicate applied inside the search call.
- ☐ Six test cases minimum, automated, three of them negative.
- ☐ Three role-scoped evaluation scores reported separately.
- ☐ All four leakage paths checked, with the log and trace stores actually grepped.
- ☐ Red-team findings reported for the system you attacked, and for your own.

Defend your result

"Our naive design answered a restricted question with clean citations. Our filtered design refuses it. Our gold set scores ____ / ____ / ____ for the three roles. We found ____ leaks in the system we attacked and ____ in our own — the one that surprised us was ____."

The Day 2 assignment, and where each piece comes from

Reference

The four tasks are designed so that finishing them produces the assignment. Nothing here is extra work — it is the same artefacts, written up for your own proof of concept rather than for the lab case.

Assignment deliverable	Where you built it
1 Architecture pattern selected for your PoC	Task 1, Step 1 — the ADR format, applied to your own case instead of the five lab cases
2 Selection rationale and trade-off analysis	Task 1 — the three deciding criteria, one rejected alternative, and the reversal cost
3 Target architecture diagram	Task 2 — <code>layers.yaml</code> and the generated target-state diagram, all nineteen layers marked
4 Storage layer design	Task 3 — every data object mapped to one of the eleven layers, with version and retention
5 Data flow diagram	Task 3 — entry, transformation, storage and exit points, including the feedback path
6 Security and access control design	Task 4 — roles, access model, retrieval filter logic, and the tool authorisation boundary
7 Scalability risks and mitigation	Task 2, Step 5 — what breaks at ten times volume, which component first, measured rather than guessed

Scored on architecture fit, completeness of layer coverage, data design quality and security rigour.

Data and references

Item	Reference
MetroPT-3 Dataset Task 1	UCI Machine Learning Repository, dataset 791. The Air Production Unit compressor of a metro train — 1,516,948 readings at 1 Hz across 15 sensors, February to August 2020, with four documented air-leak failures on 18 April, 29–30 May, 5–7 June and 15 July. File <code>MetroPT3(AirCompressor).csv</code> . archive.ics.uci.edu/dataset/791
Embedding models Task 3	<code>text-embedding-ada-002</code> and <code>text-embedding-3-small</code> are both documented at 1,536 output dimensions, which is what allows them to be mixed without error. <code>text-embedding-3-large</code> returns 3,072 by default and would fail loudly instead — a useful contrast to mention in your readout. Offline equivalent: <code>all-MiniLM-L6-v2</code> and <code>all-MiniLM-L12-v2</code> , both 384 dimensions.
Corpus Tasks 3 and 4	Generated for the exercise. A synthetic corpus is the correct choice here because retrieval quality must be measurable against a known-correct answer for every question — which is not possible with a real corpus you have not annotated.

On data hygiene. Nothing in this pack requires Atlas Copco data, and no customer or production data should be placed on a training virtual machine — particularly today, when several tasks deliberately involve access control and leakage. The scenarios are framed in Atlas Copco terms because the framing is what makes the architecture decisions concrete. The data is public or synthetic throughout.

The five sentences worth taking out of the room

- 1 The pattern is the decision with the longest half-life. A wrong metric is replaced in an afternoon; a wrong architecture is rewritten.
- 2 Answer the six selection questions before building. Data type, ground truth, latency, autonomy, access, integration — they eliminate most options quickly.
- 3 A proof of concept has three components. A production system has nineteen, and the other sixteen each exist because something failed without them.
- 4 Access control belongs inside the retrieval query. Filtering the output is not access control.
- 5 Treat model output as untrusted input at every point where it crosses into another system. It is generated text, not a verified instruction.

Before you leave today

Name, as a team, the layer you were most surprised to find missing from your own system. Write it down. That answer sets up Day 3, where retrieval and observability get built out properly — and where most of what you name today turns out to be something you can now measure.